

# 品質ゲート仕様

## 目次

|                                                                             |   |
|-----------------------------------------------------------------------------|---|
| 1 ① spec 検査( <code>review_checks.py spec &lt;fid&gt;</code> ) .....         | 4 |
| 2 ② testspec 検査( <code>review_checks.py testspec &lt;fid&gt;</code> ) ..... | 4 |
| 3 一斉レビュー支援( <code>review_checks.py report</code> ) .....                    | 4 |
| 4 総点検( <code>review_checks.py all</code> ) .....                            | 4 |
| 5 <code>guard_tests.py</code> (テスト改変の物理防止) .....                            | 6 |
| 6 <code>guard_json.py</code> (正データ破損の防止) .....                              | 6 |

LLM の成果物は、人のレビューに届く前に**決定的スクリプトの検証**を必ず通る。「NG が残る成果物を『できました』と報告してはいけない」が全フェーズ skill の共通規則。

表 1

| フェーズ       | ゲート                                      | 検知対象                                                                    |
|------------|------------------------------------------|-------------------------------------------------------------------------|
| ③ 完全性      | 抽出器(extract_fortran.py / extract_c.py)内蔵 | 2 系統カウント突合による数え漏れ                                                       |
| ① draft 後  | review_checks.py spec                    | 実在しない <code>file:lines</code> 引用、●なのに根拠なし、プレースホルダ残存(省略)、必須節欠落、原本ハッシュ不一致 |
| ② 承認依頼前    | review_checks.py testspec                | ●仕様項目のケース漏れ、①に無い SPEC-ID 参照(捏造)、宙に浮いた TC 参照、根拠の規定外表記、spec-hash 鮮度       |
| ③ freeze 前 | pytest -collect-only + マーカー突合            | ケース ID とテスト関数の過不足 (collect_results が exit 3)                            |
| ④ 完了前      | check_stubs.py                           | 空実装・NotImplementedError・TODO/FIXME(スタブ化)                                |
| ⑤ 事前       | ledger verify                            | ② stale・freeze 後のテスト改変・blocked の見落とし                                    |
| ⑥ 前        | review_checks.py all                     | 全関数の①②総点検                                                               |
| 承認の瞬間      | サーバ側で機械レビューを再実行                          | クライアント表示の改竄・競合による「NG 付き成果物の承認」                                          |
| 全編集の直後     | hooks/guard_json.py                      | AI の編集による正データ(JSON)の破損                                                  |

この結果、人のレビューは「意味が正しいか」に集中できる。「引用された行は実在するか」「トレーサビリティは揃っているか」の照合作業は機械が済ませている。

機械で取れないもの、の分担

機械検知の限界は「式は書いてあるがレガシーの意図と違う」という意味レベルのすり替え。これは次の2つが受け持つ:

1. 人の①レビュー — Confidence(●●●)と行番号根拠が付いた状態で意味を確認する
2. ⑤の突き合わせ — 独立に作られたテスト(②③系)と実装(④系)の不一致として露見する

疑わしければ ISSUE(仮説+Yes/No)へ。これが「機械が正、AI は意味づけ、裁定は人」の分担。

## 1 ① spec 検査(review\_checks.py spec <fid>)

表 1-1

| 検査        | 検知するハルシネーション/手抜き                              |
|-----------|-----------------------------------------------|
| 根拠引用の实在   | file:lines が实在するレガシー行を指しているか(存在しない行への引用 = 捏造) |
| ●の根拠必須    | Confidence ●(確認済)なのに根拠引用がない項目                 |
| プレースホルダ残存 | テンプレの穴埋めが残っている(記入の省略)                         |
| 必須節の欠落    | テンプレ(assets/templates/spec.md)が定める節の欠落        |
| 原本ハッシュ鮮度  | 仕様書が参照するレガシー原本が書き換わっていないか                     |

## 2 ② testspec 検査(review\_checks.py testspec <fid>)

表 2-1

| 検査           | 検知するもの                           |
|--------------|----------------------------------|
| トレーサビリティ(下り) | ①の●仕様項目すべてにテストケースが対応しているか(ケース漏れ) |
| トレーサビリティ(上り) | 参照している SPEC-ID が①に实在するか(仕様の捏造検知) |
| TC 参照の整合     | 宙に浮いたケース参照がないか                   |
| 期待値根拠        | 根拠の表記が規定(仕様●/人間確認/実測)に準拠しているか    |
| △未確定         | 未確定件数(approved の前提はゼロ)           |
| spec-hash 鮮度 | ①改訂後の古い②でないか                     |

## 3 一斉レビュー支援(review\_checks.py report)

① draft の一斉レビュー表 docs/spec-review.md を生成(バッチ運用の要)。概要・Confidence 内訳・未回答質問を関数ごとに並べ、人が「全部 OK / F-xxxx は修正」で返せる。

品質ゲートの観点で重要なのは、この表が機械レビューの結果を人の作業順に翻訳していること。

表 3-1

| 表の仕掛け                              | 品質上の意味                                     |
|------------------------------------|--------------------------------------------|
| 「承認できる → ISSUE あり → AI 修正待ち」の順に並べる | 人が判断できる行から消化でき、判断のいらない行を眺めさせない             |
| ✖ の行には承認ボタンを出さず「AI 修正待ち」と表示        | 機械 NG は人が承認できないことを UI で担保する(押せてしまう余地を残さない) |
| ✖ の件数リンクが仕様書ページの理由全文へ飛ぶ            | 「4 件 NG」だけ見せて中身を探させない                      |

表示の詳細は 画面設計 ― 一斉レビュー表。

## 4 総点検(review\_checks.py all)

全関数の①②を一括検査(skeleton は除外)。⑥前の総点検と再開時の健全性確認に使う。

#### 承認経路の強制(サーバ側の再検証)

承認は2つのUI(仕様書ページの承認ウィジェット・一斉レビュー表の行内ボタン)から行えるが、**検証と記録は1本**にしてある。

- 承認 POST を受けた時点で**サーバ側が機械レビューを再実行**し、NG なら承認を拒否する。ボタンの活性制御(グレーアウト)は補助であって、それだけには依存しない
- status が期待値でない(既に承認済み・他で更新された)場合も拒否する。複数の画面・複数の人が同時に触っても、状態が壊れない
- 承認の記録先はフロントマターの `reviewed-by / reviewed-date`(②は `approved-*`)で、チャット承認・ブラウザ承認のどちらでも同じフィールドに書く

hooks: 拒否と差し戻しの2系統

LLMの自制に頼らず、**仕組み**で強制する層。PreToolUseは「やらせない」、PostToolUseは「やった結果を検査して直させる」という違いがある。

表 4-1

| hook           | 種別          | 何をするか                                        |
|----------------|-------------|----------------------------------------------|
| guard_tests.py | PreToolUse  | phase が 4 / 5 のとき tests/ への Edit/Write を拒否する |
| guard_json.py  | PostToolUse | .json の編集直後に構文検証し、壊れていればエージェントに差し戻す          |

## 5 guard\_tests.py(テスト改変の物理防止)

「AIがテストを書き換えて通す」事故を防ぐ。

- `.legacy-reverse/state.json` の phase が 4 または 5 のとき、tests/ への Edit/Write を拒否
- フェーズ skill は開始時 `ledger phase-start`、終了時 `phase-end` で状態を明示する契約
- テスト側に疑義があるときの正規経路は ISSUE → 人承認 → ②③再生成(hookの迂回ではない)

## 6 guard\_json.py(正データ破損の防止)

`data/functions.json` は全スクリプトが毎回読む正データで、ここが壊れると無人バッチが途中で止まる。編集の直後に `json.loads` で検証し、失敗したら **壊れた位置(行・列)**と直し方を添えて `exit 2` で差し戻す。

- 「拒否」ではなく「差し戻し」なのは、JSONの編集自体は正当な作業だから。禁じるべきは編集ではなく、壊れたまま次に進むこと
- エージェント自身があるので、人の復旧作業を待たない
- すり抜けた場合の受け皿はデータ仕様 — 正データが壊れたときの多層防御

hookはMCP化しない(ツール呼び出しの強制ガードはhookの役割のまま)。

スタブ検知: `check_stubs.py`

④の「動いた風」を防ぐ。検知対象:

- 本体が `pass` / `...` だけの関数(空実装)
- `NotImplementedError`
- `TODO` / `FIXME` コメント

検知ありは④未完了扱い。実装しきれない場合の正規経路は spec-gap ISSUE → ①改訂。

⑤の制御信号(collect\_results.py の exit code)

表 6-1

| exit | result   | 意味                  | 後続                                                                                                              |
|------|----------|---------------------|-----------------------------------------------------------------------------------------------------------------|
| 0    | pass     | 実装率 100% かつ失敗 0     | WBS                            |
| 1    | fail     | 失敗あり                | トライアージ((a) は自走、attempt +1)                                                                                      |
| 2    | blocked  | attempt 上限 3 到達     | triage ISSUE 自動起票・WBS  ・人の裁定待ち |
| 3    | mismatch | ②のケース ID と③のマーカー不整合 | ③の突合修正(freeze 前なら③へ差し戻し)                                                                                        |



完了検証: ledger check(⑥)

フロントマター走査+ハッシュ照合で全関数を総点検し docs/completion-check.md を生成。

- ①～⑤の完了条件(データ仕様 — status 遷移)を全関数で検査
- open ISSUE ・ blocked ・ stale ・ 変更の残存を列挙
- `review_checks.py all` と併せて両方 exit 0 が⑥ pass